Homework 1

Kevin Phan

1. Generate 1000 random draws from a standard normal distribution and store the values in a vector called x.

```
set.seed(67)
x <- rnorm(1000)
```

   a. Remove all the values of this vector between 0 and 1. How many values remain in the vector? Compute the mean of the remaining values.

```
length(x[x <= 0 | x >= 1])
```

```
[1] 635
```

```
mean(x[x <= 0 | x >= 1])
```

```
[1] -0.2567413
```

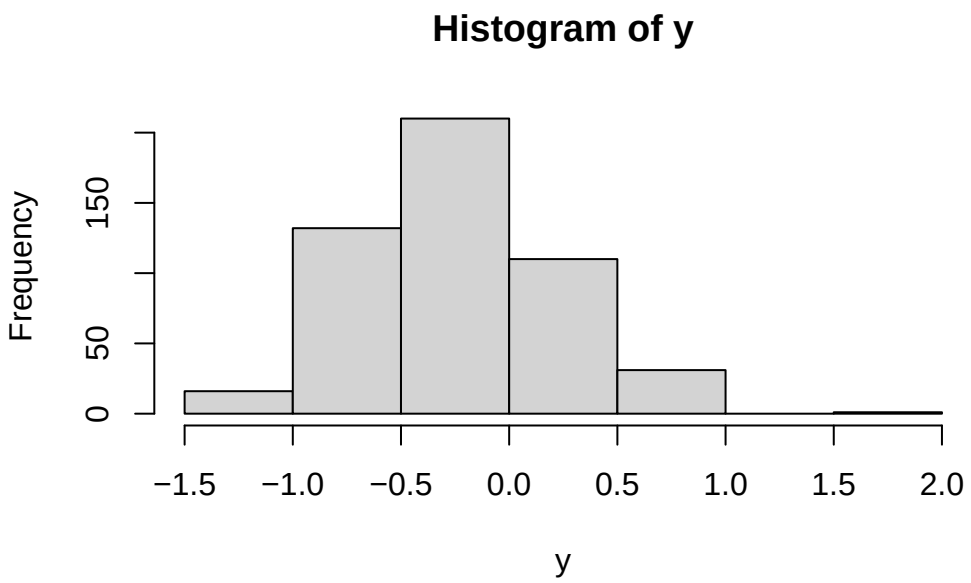
   b. Write a function that computes this mean as a function of n. Run this function once, for $n = 100$.

```
normal_mean <- function(n) {
  x <- rnorm(n)
  return(mean(x[x <= 0 | x >= 1]))
}

normal_mean(100)
```

```
[1] 0.1435681
```

   c. Using a for loop, run this function 500 times with $n = 10$. Store these 500 values in a vector and then plot a histogram of these values. Report the largest and smallest values from the 500 runs.

```r
y <- numeric(500)
for (i in 1:500) {
  y[i] <- normal_mean(10)
}

hist(y)
```




```r
max(y)
```

```
[1] 1.591957
```

```r
min(y)
```

```
[1] -1.425749
```

2. The game of craps is played as follows. First, you roll two six-sided dice; let x be the sum of the dice on the first roll. If $x = 7$ or 11 you win immediately, if $x = 2$, 3, or 12 you lose immediately, otherwise you keep rolling until either you get x again, in which case you also win, or until you get a 7, in which case you lose.

Write a program to simulate a game of craps and estimate the probability that a player wins. You can use the following snippet of code to simulate the roll of two (fair) dice:

```r
x <- sum(sample(1:6, 2, replace = TRUE))
```

```r
craps_win <- function() {
  roll_1 <- sum(sample(1:6, 2, replace = TRUE))
  if (roll_1 == 2 | roll_1 == 3 | roll_1 == 12) {
    return(FALSE)
  } else if (roll_1 == 7 | roll_1 == 11) {
    return(TRUE)
  } else {
    next_roll <- sum(sample(1:6, 2, replace = TRUE))
    while(next_roll != roll_1) {
      if (next_roll == 7) {
        return(FALSE)
      } else {
        next_roll <- sum(sample(1:6, 2, replace = TRUE))
      }
    }
    return(TRUE)
  }
}

win_pct <- function(n) {
  games <- logical(n)
  for (i in 1:n) {
    games[i] <- craps_win()
  }
  return(sum(games) / n)
}

win_pct(100)
```

```
[1] 0.52
```

```r
win_pct(1000)
```

```
[1] 0.487
```

```r
win_pct(10000)
```

```
[1] 0.4869
```

```
win_pct(100000)
```

```
[1] 0.49395
```

```
win_pct(1000000)
```

```
[1] 0.492355
```

As the number of craps games played increases, the percentage of craps games won seems to converge to around 49%. Therefore, by the law of large numbers, I estimate the probability of winning to be about 0.49.

3. Using a while loop, write a function that counts the number of times a 6-sided die must be rolled until you get a six and returns the number of rolls. Run this function a large number of times to estimate the average number of times a die must be rolled until you get a 6.

```
rolls_until_6 <- function() {
  roll <- sample(1:6, 1)
  num_rolls <- 1
  while (roll != 6) {
    roll <- sample(1:6, 1)
    num_rolls <- num_rolls + 1
  }
  return(num_rolls)
}

average_rolls <- function(n) {
  rolls <- numeric(n)
  for (i in 1:n) {
    rolls[i] <- rolls_until_6()
  }
  return(mean(rolls))
}

average_rolls(100)
```

```
[1] 5.38
```

```
average_rolls(1000)
```

```
[1] 5.813
```

```
average_rolls(10000)
```

```
[1] 5.9441
```

```
average_rolls(100000)
```

```
[1] 5.98653
```

```
average_rolls(1000000)
```

```
[1] 5.998685
```

As the number of die roll experiments increases, the average number of rolls to roll a 6 seems to converge to about 6. Therefore, by the law of large numbers, I estimate the average number of die rolls it takes to roll a 6 is about 6.

4. Write a function that computes the k-Winsorized mean of a vector (the k-Winsorized mean calculates the average of a dataset after replacing its k smallest and k largest values with their nearest remaining neighbor to minimize outlier distortion). The arguments of the function should be a vector and k.

```
k_winsorized_mean <- function(vec, k) {
  vec <- sort(vec)
  for (i in 1:k) {
    vec[i] <- vec[k + 1]
    vec[length(vec) - i + 1] <- vec[length(vec) - k]
  }
  return(mean(vec))
}
```

5. Simulate $n = 10$ observations from a chi squared distribution with 5 degrees of freedom. Compute the mean of these observations, and then repeat this process 10000 times to get 10000 means each from a sample size of 10. Compute the mean and variance of this sample of 10000 means (theoretically, what *should* the mean and variance be? Is your answer close?).

```r
chi_means <- numeric(10000)
for (i in 1:10000) {
  chi_means[i] <- mean(rchisq(10, 5))
}

mean(chi_means)
```

```
[1] 5.0119
```

```r
var(chi_means)
```

```
[1] 1.014192
```

I calculate a mean of $\overline{x} = 5.0119$ *and a variance* $s^2 = 1.0141$ *. This aligns with what is expected* ($\mu = k = 5$ *and* $\sigma^2 = \frac{2k}{n} = 1$).

Next, using the function that you wrote in question 4, generate $n = 10$ observations from a chi squared distribution with 5 degrees of freedom and calculate the k-Winsorized mean with $k = 1$. Repeat this 10000 times to get 10000 1-Winsorized means. Compute the mean and variance of these 10000 1-Winsorized means. Then repeat this for $k = 2$, 3, and 4-Winsorized means and compute the mean and varaince of these sample means. Show your results in a table with three columns k, mean, and variance. Also, display your results with the value k on the x-axis and the mean on the y-axis with a red horizontal line representing the true theoretical mean.

```r
k_winsorized_chi_means <- function(k) {
  winsorized_means <- numeric(10000)
  for (i in 1:10000) {
    winsorized_means[i] <- k_winsorized_mean(rchisq(10, 5), k)
  }
  return(winsorized_means)
}

one_winsorized_chi_means <- k_winsorized_chi_means(1)
two_winsorized_chi_means <- k_winsorized_chi_means(2)
three_winsorized_chi_means <- k_winsorized_chi_means(3)
four_winsorized_chi_means <- k_winsorized_chi_means(4)

winsorized_table <- data.frame(k = c(1, 2, 3, 4),
                               mean = c(mean(one_winsorized_chi_means),
                                        mean(two_winsorized_chi_means),
                                        mean(three_winsorized_chi_means),
```

```
                                        mean(four_winsorized_chi_means)),
                                variance = c(var(one_winsorized_chi_means),
                                             var(two_winsorized_chi_means),
                                             var(three_winsorized_chi_means),
                                             var(four_winsorized_chi_means)))

winsorized_table
```

```
  k     mean  variance
1 1 4.818129 0.9894166
2 2 4.646050 1.0324104
3 3 4.531976 1.0688616
4 4 4.475185 1.2315979
```

```
plot(x = winsorized_table$k,
     y = winsorized_table$mean,
     type = 'l',
     ylim = c(4, 5),
     xlab = 'k',
     ylab = 'mean')

abline(h = 5, col = 'red')
```